Parsek Roadmap

Philosophy

Parsek is a **git-like recording system** for KSP missions. You record flights sequentially, commit them to a timeline, and they play back automatically as ghost vessels while you fly new missions. There is one timeline, recordings are immutable commits, and the game always moves forward.

Like git, you can go back to any earlier point and start new work. Existing recordings don't change — they play out as ghosts alongside your new missions. There is no branching, no state reversal, and no paradoxes. Conflicts are prevented through resource budgeting: committed recordings have already claimed their costs, so the player can only spend what's actually available.

Time travel paradoxes are avoided by two invariants: **causality** (events are always processed in time-axis order) and **additivity** (the timeline is append-only — committed recordings and game actions can be added but never deleted). The world state at any point in time is fully determined by the ordered sequence of committed events before it. When a new event is inserted into the timeline (e.g., a recording committed at an earlier UT after a rewind), all derived resource values — available science, funds, reputation — are recalculated from scratch across the full timeline. Events themselves are immutable; the computed state that flows from them is not stored but recomputed whenever the event set changes.

Completed

v0.3 — Foundation

Initial release. The core loop shipped end-to-end: record a flight, commit it, rewind, see it play back as a ghost alongside new missions.

Recording & playback: Position recording with adaptive sampling and geographic coordinates. Kinematic ghost playback with opaque vessel replicas. Orbital recording with analytical Keplerian orbits. Auto-recording on launch and EVA. Chained recordings across vessel lifecycle events (undock, EVA, dock). Context-aware merge dialog (Merge to Timeline / Discard). Vessel persistence with deferred spawn, crew reservation, and resource deltas.

Ghost visual fidelity: Comprehensive part event types recorded and replayed on ghost vessels — decoupling, staging, parachutes, engines (with FX), solar panels, antennas, radiators, landing gear, lights, cargo bays, fairings, RCS (with FX), docking/undocking, inventory parts.

Career mode: Milestones (tech research, part purchases, facility upgrades, contracts, crew changes) captured as immutable timeline commits. Resource budget computed on-the-fly from recordings + milestones. Epoch isolation to prevent abandoned timeline branches from leaking. Resource deduction on revert so KSP's top bar reflects available resources. Action blocking to prevent re-researching committed tech or re-upgrading committed facilities. Per-recording rewind saves with quicksave-based timeline restoration. Rewind UI with confirmation dialog, resource reset, crew re-reservation, and action replay. Fast-forward to advance past committed recordings.

Recordings Manager UI with sortable columns, per-recording loop/hide, rewind/fast-forward, and status indicators.

Design: `docs/dev/done/design-restore-points.md`, `docs/dev/done/design-going-back-in-time.md`

v0.4 — Visual Fidelity & Polish

Orbital rotation fidelity, watch camera, heat effects, KSC scene playback, UI polish, and code refactoring.

Orbital rotation: Ghost vessels in orbital segments preserve their recorded attitude instead of always facing prograde. Rotation stored relative to the orbital velocity frame at the on-rails boundary and reconstructed from the Keplerian orbit. SAS-locked orientations hold correctly throughout the orbit. PersistentRotation mod detected and supported — spinning vessels reproduced during playback.

Watch camera: Camera follows a ghost vessel during playback. Automatically recenters on visible parts after separation events.

Ghost visuals: Re-entry heating FX with mesh-surface fire particles. Heat shield ablation, smoke and spark FX on decouple/destroy. Fairing ghost visuals. EVA kerbal facing fix. Explosion visual effect on impact with camera hold.

KSC playback: Ghosts visible in KSC scene with overlap support and distance-based part event culling.

UI & usability: Per-save settings panel (auto-record, sampling thresholds). Recording stats (max altitude, max speed, max G, distance, duration). Non-revert "Commit Flight" button. Two-phase parachute deploy. Auto-loop with per-recording interval controls. Recording groups with multi-membership. Time warp visual cutoffs and deferred spawn queue. Context-aware rewind button.

Code health: Major refactor reducing coupling between modules. Sandbox rewind fix.

Design: `docs/dev/done/design-orbital-rotation.md`, `docs/dev/done/design-camera-follow-ghost.md`

v0.5 — Recording System Redesign

Comprehensive redesign of the recording and playback systems. Multi-vessel sessions, ghost chain paradox prevention, spawn safety, time jump, rendering zones, and ghost visual hardening.

Design: `docs/parsek-flight-recorder-design.md` (consolidated design document)

Recording model:

- Segment boundary rule — only physical structural separation creates new segments; controller changes, part destruction without splitting, and crew transfers are SegmentEvents within a continuing segment
- Crash coalescing — rapid split events grouped into single BREAKUP events
- Environment taxonomy — classification (Atmospheric, ExoPropulsive, ExoBallistic, SurfaceMobile, SurfaceStationary) with hysteresis
- TrackSections — typed trajectory chunks tagged with environment, reference frame, and data source
- Reference frames — ABSOLUTE for physics, ORBITAL_CHECKPOINT for on-rails, RELATIVE for anchor-vessel proximity

Multi-vessel sessions:

- Recording tree (DAG) — multi-vessel missions recorded as a single unit with split/merge tracking across undock, EVA, dock, joint break, and breakup events
- Background vessel recording — all vessels in the physics bubble sampled at proximity-based rates with full part event capture
- Booster/debris recording — automatic tree promotion on staging with debris TTL
- Highest-fidelity-wins merge — overlapping Active/Background/Checkpoint data merged per vessel
- Flag planting recording and playback

Ghost chain paradox prevention:

- Committed recordings that interact with pre-existing vessels cause those vessels to become ghosts from rewind until the chain tip resolves
- Intermediate spawn suppression — multi-link chains only spawn at the tip
- Ghost conversion — real vessels despawned and replaced with ghost GameObjects during chain windows

- PID preservation — chain-tip spawns preserve the original vessel's persistent ID for cross-tree chain linking
- Ghosting trigger taxonomy — structural events, orbital changes, and part state changes trigger ghosting; cosmetic events do not

Spawn safety:

- Bounding box collision detection — spawn blocked when overlapping with loaded vessels
- Ghost extension — ghost continues on propagated orbit/surface past recording end while spawn is blocked
- Trajectory walkback — for immovable blockers, walks backward along recorded trajectory to find a valid spawn position
- Terrain correction — surface spawns adjusted for terrain height changes between recording and playback
- KSC exclusion zone — 50m safety radius around launch pad and runway
- Spawn-die-respawn prevention (3-cycle limit), spawn abandon flags, retry limits (150 frames)
- Snapshot situation correction (FLYING→LANDED for landed terminals, FLYING→ORBITING for orbital terminals)
- Non-leaf tree recording spawn suppression, unsafe snapshot situation blocking
- Dead crew protection on spawn (strip dead crew, abandon all-dead spawn)

Time jump:

- Relative-state time jump — discrete UT skip preserving rendezvous geometry across ghost chain windows
- Epoch-shifted orbits — orbital elements recomputed at the new UT from state vectors

Rendering & performance:

- Distance-based zones — Physics, Visual, Beyond — with zone-aware playback and part event gating
- Ghost soft caps — configurable thresholds with priority-based despawning, reduced fidelity (75% renderer culling), simplified orbit-line mode
- Terminated chain early-out — fully-terminated trees and chains skip per-frame evaluation

Ghost visual hardening: Variant textures and materials (TEXTURE/MATERIAL/GAMEOBJECT rules), damaged wheel filtering, fairing meshes with procedural truss and nosecone caps, SRB nozzle glow (FXModuleAnimateThrottle), engine shrouds with variant awareness, initial state seeding for all 16 tracking set types. Compound part visuals (fuel lines, struts). Plume and smoke trail fixes (KSPParticleEmitter native control, emission module disabled). Control surfaces, robotics/servo detection, cabin lights, animation-based deployables. RCS debounce (8-frame threshold). Part separation smoke/spark FX. Lingering particle systems on ghost despawn.

Ghost world presence: CommNet relay via antenna specs with correct combination formula. Ghost labels. ProtoVessel-based map presence — tracking station entries, orbit lines, navigation targeting, ghost icon click popup (Set Target / Watch / Focus). Harmony guard rails (27 checks across 9 files, 4+ patches). Orbit segment updates on SOI transitions. Tracking station NRE protection.

Real Spawn Control: Proximity-based UI for warping to when nearby ghost craft become real vessels. Per-craft warp buttons, sortable columns, countdown display, screen notifications. Countdown column in Recordings Manager.

Per-phase looping: Chain recordings split at atmosphere/altitude/SOI boundaries. Tree recordings split by optimizer at environment boundaries. Auto loop range trims boring bookends. Debris loop sync via LoopSyncParentIdx.

Watch mode hardening: Zone-hide exemption for watched ghosts (no 120km cutoff). Hold timer with retry for chain transitions. Auto-follow through tree branching with PID-matched descent. FF watch transfer. Camera bridge anchors at cycle boundaries. Distance guard (100km rendering-safe limit). Orbital recording exemption from distance cutoffs.

Rewind/spawn hardening: Future vessel strip (PRELAUNCH and all types via quicksave PID whitelist). Spawn state reconciliation after strip. EVA vessel removal with crew rescue. Localization key resolution at all comparison sites. Spawn position from snapshot (not trajectory endpoint). EVA endpoint spawning. Ladder state stripping. Orbital vessel spawn via state vector orbit construction.

UI: Fast-forward redesigned as instant UT jump. Simplified merge dialog. Compressed Status column (merged countdown). Group header aggregate stats. Recording group unique naming. In-game test runner (90 tests, Ctrl+Shift+T).

Code health: GhostPlaybackEngine extraction (zero Recording references, IPlaybackTrajectory interface). ChainSegmentManager extraction. ParsekPlaybackPolicy event-driven architecture. Comprehensive log audit (92% output reduction). 4600+ xUnit tests. PartStateSeeder unification. GhostBuildResult consolidation.

v0.6 — Game Actions System

Full redesign of milestone capture, resource budgeting, and action replay. Standalone resource ledger tracking every economic event on the timeline.

Design: `docs/parsek-game-actions-and-resources-recorder-design.md`

Architecture: Sidecar files capture raw data during flight; extraction to ledger on commit. Two-phase recalculate+patch on warp exit. Unified recalculation walk from UT=0 with two-tier module dependency ordering. KSP UI patched to display available funds (not gross balance).

Resource modules (8):

Module	Key mechanics
Science	Immutable awarded values + derived effective values, per-subject caps via full recalculation sorted by UT
Funds	Seeded balance from save, reservation system prevents overspending across rewinds, vessel build costs as recording-associated spendings
Reputation	Non-linear gain/loss curve, nominal vs effective values
Milestones	Once-ever binary cap, chronological priority, first-tier feeds funds and reputation
Kerbals	IResourceModule participant, reservation from UT=0 as continuous block, temporary vs permanent loss, replacement chains, stand-in generation, managed-kerbal filtering
Facilities	Upgrade/destruction/repair schemas, visual state management during fast-forward, KSP state patching, deferred replay for missing facilityRefs
Contracts	UT=0 reservation, accept/complete/fail/cancel lifecycle, parameter progress tracking, deadline failure detection
Strategies	UT=0 reservation, transform layer between first-tier and second-tier modules, commitment rates

Paradox prevention: No-delete invariant, spending reservation (science/funds), UT=0 reservation (kerbals/contracts/strategies). Milestone path qualification. Budget deduction clamping. Chain gap closure for game state events.

Crew lifecycle: Crew reservation via internal managed state (Harmony patches for KSP roster validation and astronaut complex count). Crew swap in both flight and KSC spawn snapshots. Orphaned crew rescue after vessel strip. Dead crew spawn protection. Kerbal rescue detection. Managed-kerbal event filtering.

Validated across all game modes: Sandbox (no resources), Science (science-only), Career (full complexity). 4621 tests passing.

v0.7 — Timeline

Unified chronological view of all committed career events, replacing the Game Actions window.

Design: `docs/parsek-timeline-design.md`

Timeline window: Flat chronological list of career events sorted by UT. Current-UT divider separates past (full color) from future (dimmed). Two significance tiers: Overview (mission structure — launches, milestones, contracts, facilities) and Detail (all resource transactions). Three source filters: Recordings, Actions (deliberate player choices), Events (gameplay consequences). Vessel-level telemetry (part events, segment events) stays in the Recordings Manager.

Entry display: EVA-aware (`EVA: Jeb from Mun Lander (MET 5s)` , `Board: Jeb (Mun Lander)`). Spawn at EndUT with VesselSituation (`Spawn: Vessel (Landed on Mun)`). Mission Elapsed Time on launches (KSP calendar). Humanized science subjects, tech nodes, milestones, strategy names. Color-coded: green = earnings, red = penalties, light blue = player actions, white = recordings.

Operations: Rewind (R) and Fast-Forward (FF) buttons on recording entries. GoTo cross-link opens Recordings Manager, unhides recording, expands parent groups, scrolls to target.

Data model: 26 entry types (2 recording lifecycle + 23 game actions + 1 legacy). `TimelineBuilder` with 3 collectors + stable UT sort. `IsPlayerAction` classification. Game-mode aware (career/science/sandbox). 4870 tests passing (53 timeline-specific).

Location context: Recordings become location-aware — body, biome, situation, and stock launch site name captured at recording start and end. Timeline shows location-enriched entries ("Launch: Vessel from Launch Pad on Kerbin", "Spawn: Vessel (Landed at Midlands on Mun)"). Sortable Site column in Recordings Manager with UT tiebreak. Prerequisite for logistics routes and async multiplayer.

Phase 11: Resource Snapshots

Recordings capture physical resource manifests at recording start and end.

Resource manifests (implemented): `ExtractResourceManifest` walks vessel snapshot PART > RESOURCE nodes, sums by resource name. `StartResources` / `EndResources` fields on Recording, serialized as additive RESOURCE_MANIFEST ConfigNode. Captured at 8 boundary sites (recording start/stop, chain boundaries, breakup, background splits, optimizer splits/merges). EC and IntakeAir excluded (environmental noise). Dock target vessel PID (`DockTargetVesselPid`) captured at dock boundaries for route endpoint identification. Hover tooltip in Recordings Manager shows per-resource start-to-end with delta.

Inventory manifests (implemented): KSP 1.12 `ModuleInventoryPart` items (stored parts in cargo containers). `ExtractInventoryManifest` walks MODULE > STOREDPARTS > STOREDPART nodes. `InventoryItem { count, slotsTaken }` struct + vessel-level `totalInventorySlots` . Same capture sites as resources.

Crew manifests (implemented): Crew composition by trait (Pilot/Scientist/Engineer/Tourist). Route delivery uses generic kerbals (separate from crew reservation system). Same capture pattern.

Phase 11.5: Recording Optimization & Observability

Optimization pass before logistics routes add many long-lived looped recordings. A long career with dozens of missions will accumulate significant disk and memory footprint — this phase makes that measurable and then reduces it.

Observability (first)

Instrument the system so size/performance problems are visible during playtesting, without introducing behavior changes:

- **Per-save storage report** — total disk size of Parsek data (sidecar files + .sfs metadata), broken down by recording. Accessible from Settings > Diagnostics.
- **Per-recording stats** — point count, part event count, orbit segment count, sidecar file sizes (.prec, _vessel.craft, _ghost.craft). Visible in Recordings Manager tooltip or detail view.
- **Playback budget** — per-frame timing for ghost positioning, part event application, zone evaluation. Logged via `ParsekLog.VerboseRateLimited`, surfaced in diagnostics.
- **Memory footprint estimate** — loaded trajectory point count, loaded snapshot count, ghost mesh count. Logged at scene load and on-demand.
- **Recording growth rate** — logged during recording: points/second, events/second, estimated file size at current rate.

Optimization (after measurement)

Based on what the observability pass reveals, candidates include:

- **Shorter key names** in .prec trajectory serialization (e.g., `u` instead of `ut`, `la` instead of `lat`)
- **Compact numeric encoding** — fixed decimal places instead of round-trip `"R"` format where precision isn't needed
- **Vessel snapshot deduplication** — ghost visual snapshot often duplicates vessel snapshot; store once and reference
- **Part event name deduplication** — index table for repeated part names in event lists
- **Trajectory point thinning** — post-commit pass that removes redundant points (straight-line segments, stationary holds) beyond what adaptive sampling already does
- **Optional gzip compression** for sidecar files
- **Lazy loading** — don't load trajectory data for recordings outside the current playback window

This phase is explicitly measurement-first: add observability, playtest a full career, identify the actual bottlenecks, then optimize the ones that matter. No speculative optimization.

Phase 12: Looped Transport Logistics

Automated supply routes realized through Parsek's existing loop mechanic. Fly a cargo run once, loop the recording, each iteration is a supply delivery.

- **Route definition** — a recording with declared origin (Phase 10) and destination (Phase 10) plus resource/inventory/crew manifests (Phase 11)
- **Three cargo types** — resources (LF, Ox, Ore, etc.), inventory items (stored parts in cargo containers), and crew (generic kerbals by trait)
- **Delivery logic on loop completion** — recording completion triggers cargo transfer to nearest vessel within range
- **Round-trip support** — initially two separate looped recordings (outbound loaded, return empty); eventual round-trip recording mode
- **Time scaling** — deliveries at realistic intervals with UI showing "next delivery in: 3d 4h"
- **Visual presence** — ghost supply ships actually fly (as ghosts) on every loop iteration; only approach/departure bubbles spawn visible ghosts, transit is invisible (the boring middle)

Every supply ship is a replay of a real mission the player flew — more immersive than abstracted route systems, more performance-demanding, but achievable within the existing architecture.

Phase 13: Cooperative Async Multiplayer

Multiple players contribute recordings to a shared timeline. The Kerbal system feels populated with vessels flying and bases being built. All players share one game actions timeline — science, funds, reputation, contracts, and kerbals are pooled.

Gloops Extraction (Phase 13 Prerequisite)

Extract the ghost playback engine into a separate assembly (`Gloops.dll`) within the same repository. This provides build-time boundary enforcement and defines the `.gloop` file format needed for recording export/import. See `docs/dev/gloops-recorder-design.md` for the full design.

Extraction timing rationale: The engine boundary already exists (IPlaybackTrajectory, IGhostPositioner, GhostPlaybackEngine with zero Recording references). Extracting earlier than Phase 13 adds overhead without user benefit — new features through Phase 12 still touch engine code, and doing that across assemblies adds friction. Phase 13 is the natural trigger because recording export/import requires a standalone file format (`.gloop`), which is exactly what the extraction produces.

Extraction scope:

- Separate .csproj in the same repo (not a submodule yet)
- 17 files move to Gloops, 2 files need pre-extraction splitting
- Pre-extraction refactors: split `GhostPlaybackLogic.cs` (engine vs. policy), extract recorder from `FlightRecorder.cs` , `ParsekLog` abstraction
- Parsek becomes a consumer of the Gloops API

Standalone Gloops mod (post Phase 13, if demand exists):

- Split into separate repository / submodule
- Content pack system for ambient world activity (KSC traffic, scenery)
- Standalone UI (loop manager, pack toggles, settings)
- Custom mesh support for non-vessel content

Recording Export/Import

Prerequisite for multiplayer. Share recordings as standalone `.gloop` archive files. Export bundles trajectory data, vessel snapshot, and metadata. Import validates, assigns new IDs, and injects into current save. Handles missing parts gracefully (warn, skip, or substitute).

Shared Timeline

- **Shared recordings folder** — local or cloud-synced where players drop recording files
- **Import/merge on load** — Parsek scans the shared folder for new recordings and merges them into the local timeline as ghosts; purely additive, no conflict with live game state
- **Player identity** — each recording tagged with a player ID for filtering, color-coding, and attribution
- **Launch pad allocation** — different players claim different launch sites (Phase 10); each player's recordings originate from their claimed sites
- **Version tolerance** — additive recording format means players on different Parsek versions can share recordings safely

Players never need to be online simultaneously. Fly missions, export recordings. Others import whenever they play. The timeline converges over time — correspondence chess, not real-time multiplayer.

Phase 14: Competitive Play & Space Race

Player boundaries that enable competitive multiplayer. Each player has their own game actions timeline (separate science, funds, reputation, contracts, kerbals). Only things with visible effect on the common game world are shared — vessel recordings play as ghosts in everyone's game, but each player's economy is independent.

Player Boundaries

- **Per-player game actions timeline** — each player's resource ledger is independent; importing another player's recordings adds their ghosts to the world but does not affect the local player's science, funds, or reputation
- **Shared world layer** — vessel recordings (trajectories, part events, ghost chains) are shared and visible to all players; the physical world is common, the economic world is separate
- **Milestone visibility** — all players can see each other's milestones (first orbit, first Mun landing) but milestones only feed into the achieving player's resource pipeline

Space Race

- **Pre-built recording packs** — a set of recordings representing an AI space program's progression (first orbit on Day 30, Mun landing on Day 150, etc.); hand-crafted or community-contributed
- **Milestone racing** — Parsek already tracks milestones; space race is a comparison UI showing both programs' achievements side by side
- **Difficulty scaling** — different packs for different skill levels
- **Community races** — players export their full campaign as a space race opponent ("Can you beat Scott Manley's career run?")

The Dependency Chain

```
v0.3–v0.5: Core Recording System (COMPLETE)
    │  Recording, playback, loops, ghost chains, spawn safety,
    │  time jump, rendering zones, multi-vessel sessions
    │
    ▼
v0.6: Game Actions System (COMPLETE)
    │  Science, funds, reputation, kerbals, contracts,
    │  facilities, strategies, reservations, recalculation
    │
    ▼
Phase 9: Timeline (v0.7 ✓)
    │  Unified chronological view of all committed events,
    │  significance tiers, filtering, rewind from timeline
    │
    ▼
Phase 10: Location Context (v0.7 ✓)
    │  Recordings know WHERE they start/end (body, biome, situation)
    │
    ▼
Phase 11: Resource Snapshots (done)
    │  Recordings know WHAT they carry
    │
    ▼
```

```
Phase 11.5: Recording Optimization & Observability
    │  Measure disk/memory/perf, then optimize what matters
    │
    ▼
Phase 12: Looped Transport Logistics
    │  Routes = looped recordings with resource delivery
    │
    ▼
Gloops Extraction ─── Extract ghost engine to separate assembly,
    │                   define .gloop file format
    ▼
Phase 13: Cooperative Async Multiplayer
    │  .gloop export/import, shared folder, player identity
    │
    ▼
Phase 14: Competitive Play + Space Race
    │  Per-player game actions, milestone racing, opponent packs
    │
    ▼
Phase 15: Mod Compatibility
    │  Kerbal Konstructs, KSC Switcher, Extraplanetary Launchpads,
    │  off-world construction, modded launch sites via reflection
    │
   ...
Standalone Gloops Mod (if demand exists)
       Separate repo, content packs, standalone UI, custom meshes
```

Phase 15: Mod Compatibility

All prior phases target stock KSP (including Making History DLC). This phase adds support for popular mods via reflection-based detection and API integration. Only attempted after the stock experience is stable and fun.

- **Kerbal Konstructs / KSC Switcher** — detect custom launch sites, capture site name at recording start, display in timeline and Recordings Manager
- **Extraplanetary Launchpads** — detect EL-spawned vessels (builder PartModules: `ELLaunchpad`, `ELDisposablePad`, `ELSurveyStation`), capture pad name, handle empty `launchedFrom` field
- **Off-world construction mods** — detect vessels built outside KSC, tag recordings with construction origin
- **Mod detection pattern** — `AssemblyLoader.loadedAssemblies` (already used for PersistentRotation, RemoteTech). Reflection for API access. Graceful degradation when mods are absent.
- **Compatibility testing** — CustomBarnKit, Strategia, Contract Configurator (see T43)

Deferred

Planetarium.right drift compensation for long orbital segments

KSP's inertial reference frame may drift over very long time warp durations. This could cause ghost orientation mismatch for interplanetary transfer segments. Needs empirical measurement first — if drift is sub-degree for typical segment lengths, no fix needed.

Crash-safe pending recording recovery (partial)

Commit crash window closed (sidecar files flushed immediately). Remaining gap: stashed-but-not-yet-committed recordings (merge dialog pending) still live only in RAM. Solution: write a pending manifest file when a recording is stashed, auto-recover on next load if the recording wasn't committed or discarded. Low priority — remaining gap is stash→commit only.

Additional part event coverage

- Two-phase engine startup (spool-up animations on some engines)
- Engine plate covers / interstage fairings (partially fixed — mesh cloned but positioning may be wrong)

Recording file size optimization

Shorter key names, compact numeric encoding, optional compression, event name deduplication. Becomes important at scale (many recordings, long missions).

Ghost soft cap completion

LOD culling for distant ghost meshes. Ghost mesh unloading outside active time range. Particle system pooling for engine/RCS FX. Benchmark with synthetic stress tests.

Kerbal reservation refactor (T44)

Replace `rosterStatus = Assigned` workaround with Parsek-internal state + Harmony crew dialog filtering. Would eliminate 2 workaround patches and ~27 KSP warnings per session. Low priority — current workaround is functional.

Hyperbolic escape orbit line rendering (189b)

Ghost escape orbits clip at finite distance (~12,000 km). Active vessels show full escape trajectory via PatchedConicSolver which ghosts don't have. Needs custom LineRenderer or orbit line extension.

Out of Scope

- **Racing modes or lap timing**
- **AI playback or autopilot**
- **Real-time multiplayer synchronization** — Parsek's multiplayer model is async (Phases 13–14). No shared physics simulation, no lockstep networking.
- **Taking control of recorded vessels** — jumping into a ghost mid-playback creates unresolvable paradoxes (recording future events, reserved crew, applied resource deltas). The complexity is not worth the payoff.
- **Timeline branching or alternate histories**